

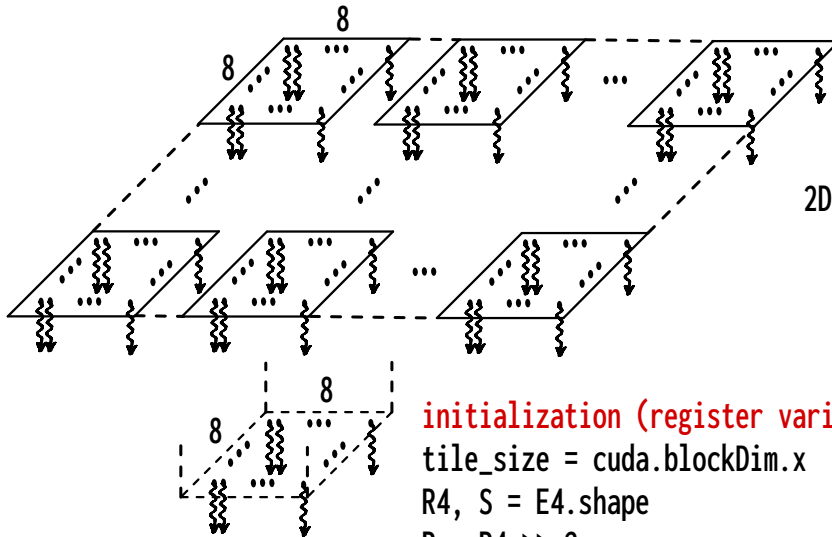
```
@cuda.jit(void(float32[:, :], float32[:, :]))
```

```
had4_numba_cuda_job_float32[bpg, tpb](E4, H4E4)
```

```
# tile_size = 8
```

```
# bpg_y = (R + tile_size - 1) // tile_size, bpg_x = (S + tile_size - 1) // tile_size
```

```
# bpg = (bpg_x, bpg_y), tpb = (tile_size, tile_size)
```



2D grid of thread blocks

initialization (register variables of each thread):

```
tile_size = cuda.blockDim.x
```

```
R4, S = E4.shape
```

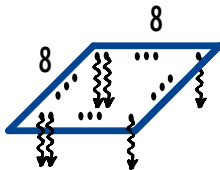
```
R = R4 >> 2
```

```
R2 = R << 1
```

```
R3 = R2 + R
```

```
row = cuda.blockIdx.y * tile_size + cuda.threadIdx.y
```

```
col = cuda.blockIdx.x * tile_size + cuda.threadIdx.x
```



copying real and imaginary parts of quaternions to register variables:

```
if row < R and col < S:
```

```
    e4_0 = E4[row, col]
```

```
    e4_1 = E4[row + R, col]
```

```
    e4_2 = E4[row + R2, col]
```

```
    e4_3 = E4[row + R3, col]
```



computing sums and differences of block pairs (0, 1) and (2, 3):

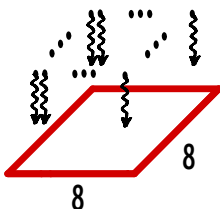
```
if row < R and col < S:
```

```
    s0 = e4_0 + e4_1
```

```
    s1 = e4_2 + e4_3
```

```
    d0 = e4_0 - e4_1
```

```
    d1 = e4_2 - e4_3
```



computing results - sum of sums, sum of differences, difference of sums, difference of differences - and storing in output array H4E4:

```
if row < R and col < S:
```

```
    H4E4[row, col] = s0 + s1
```

```
    H4E4[row + R, col] = d0 + d1
```

```
    H4E4[row + R2, col] = s0 - s1
```

```
    H4E4[row + R3, col] = d0 - d1
```